

מדריך להגנה בעל-פה על הפרויקט

איך נראית ההגנה, מה צריך להראות, ועל מה ניתן הציון

זוהי שיחה קצרה (כ-10 עד 15 דקות) על הפרויקט שלכם. המטרה היא שתראו שאתם תכננתם את המערכת, ושהקוד שקלוד בנה באמת נובע מהתכנון הזה.

אנחנו לא מעריכים את איכות הקוד שקלוד כתב, ולא נבקש מכם להסביר איך הקוד עובד שורה אחר שורה. זה לא קורס תכנות. אנחנו בודקים שיש קשר אמיתי ועקיב בין מה שתכננתם לבין מה שפותח, שאתם מסוגלים לזהות את התכנון שלכם בתוך הפרויקט, ושאתם מבינים את הארכיטקטורה ברמה הכללית.

למה הכוונה בפועל: אתם צריכים להיות מסוגלים להסתכל על הפרויקט ולהגיד "זו מחלקה (class), היא מתאימה לישות הזו בדיאגרמה שלי, והיא ממופה לטבלה או למסך הזה." לא תתבקשו להסביר איך מתודה מומשה או לקרוא קוד C# בקול.

חלוקת הציון

ציון הפרויקט מורכב משלושה רכיבים:

רכיב	משקל	על מה הוא ניתן
עמידה בדרישות	~50%	תכנות 3 מתוך 6 ה-Use Cases שבחרתם בשלב העיצוב, בתוספת מסכי CRUD מלאים (יצירה, צפייה, עדכון ומחיקה) עבור ישות אחת, מסך התחברות (login) ותפריט המוביל למסכים השונים, כשהכול עובד בפועל.
הגנה בעל-פה	~50%	התשובות שלכם לשאלות שבהמשך, והיכולת להראות שהתכנון שלכם והקוד תואמים ושאתם מבינים את הארכיטקטורה.
בונוס	עד 10%+	מעבר לנדרש (ראו סעיף הבונוס למטה). מתווסף מעל הציון.

הציון מתחלק שווה בשווה בין מה שבניתם לבין ההבנה שלכם. ההגנה בעל-פה אינה בודקת מחדש את הקוד, אלא את הקשר בין התכנון לפיתוח ואת ההבנה הארכיטקטונית, ולכן ניתן משקל זהה לעמידה בדרישות.

מה נחשב use case שעובד

use case ש"עובד" אינו רק מסך שנפתח. כדי שייחשב כהושלם, עליכם לטפל בכל מרכיביו, לא רק להראות שהמסך מוצג:

- **ולידציה ובדיקות תקינות נתונים:** שדות חובה, ערכים חוקיים, טיפוסים נכונים, והודעות שגיאה ברורות כשמשהו לא תקין.
 - **טיפול במקרי קצה:** מזהה שלא נמצא, קלט ריק, ערכים כפולים וכדומה.
 - **הזנת נתונים נוחה וניווט קל:** אפשר להזין את הנתונים בלי להיתקע, ולנוע בין השדות והמסכים בצורה הגיונית.
- הקדישו זמן לוודא שכל הזנת הנתונים עובדת היטב מקצה לקצה. use case שהמסך שלו נפתח אבל מקבל קלט לא תקין, קורס על ערך חסר, או קשה לתפעול, אינו נחשב כהושלם.

מה אתם צריכים להיות מסוגלים להראות

1. **התכנון שלכם והקוד תואמים.** אתם יודעים לנוע בין הדיאגרמות (UML/ERD) לבין הפרויקט הרץ, בשני הכיוונים, בלי ללכת לאיבוד.
2. **אתם מבינים היכן התכנון והתוצאה התרחקו זה מזה, ומה היה חסר בתכנון.** קלוד תמיד משנה משהו, והפיתוח כמעט תמיד חושף פערים (דרישה, ישות, או שדה שלא חשבתם עליהם). אתם יודעים היכן המימוש שונה מהתכנון המקורי, מה התברר כחסר, ולמה.
3. **הבנה בסיסית של הארכיטקטורה.** אתם יכולים להסביר במילים פשוטות איך האפליקציה בנויה בשכבות: מסכים, רשימות בזיכרון, בסיס הנתונים, והקשרים ביניהם.
4. **בעלות.** בקבוצה, כל חבר/ה יכול/ה לדבר על החלק שהוא/היא תכננו.

כנות עובדת לטובתכם. "כאן הקוד יצא שונה מהדיאגרמה שלי, כי..." היא תשובה טובה. "הכול תואם בדיוק בכל מקום" בדרך כלל אומר שלא באמת הסתכלתם.

איך ההגנה מתנהלת

- הביאו מחשב נייד עם **שלושה דברים פתוחים:** הדיאגרמות שלכם (Use Case Diagram והמפרטים, Class Diagram, ERD), **הקוד**, והאפליקציה **הרצה**.
- ודאו שהמחשב הנייד **טעון**, או הביאו מטען.
- אם בסיס הנתונים שלכם רץ מקומית (SQL Server מקומי), ודאו שהוא **מותקן ופועל** על המחשב שתביאו, כך שהאפליקציה עובדת מקצה לקצה.
- בלי שקפים, בלי מצגת. פשוט נדבר יחד על הפרויקט.
- נבקש מכם לנווט בזמן אמת: לפתוח קובץ, להצביע על מסך, לעקוב אחרי תהליך.
- **קבוצות:** כל חבר/ה יישאל/תישאל על use case או ישות אחרים, אז הגיעו מוכנים כשכל אחד/ת שולט/ת בחלק שלו/ה מקצה לקצה.

תרשים המצבים (State Diagram) אינו חלק מההגנה הזו, ויוגש בנפרד. עם זאת, אם אחד ה-Use Cases שפיתחתם קשור לתרשים המצבים, ציינו זאת והראו כיצד הוא בא לידי ביטוי במערכת הרצה.

כל השאלות מופיעות למטה. אי אפשר באמת "לדחוס" אותן ברגע האחרון, כי או שתכננתם את המערכת או שלא.

השאלות שעליכם להיות מוכנים אליהן

השאלות הבאות הן **קווים מנחים** וסוגי השאלות שנשאל, לא רשימה סגורה. אנחנו רשאים לשאול כל שאלה על הפרויקט שלכם, גם כזו שאינה מופיעה כאן. הרשימה נועדה להראות לכם למה להתכונן, לא להגביל אותנו.

א. מהתכנון לקוד (איתור התכנון שלכם בתוך הפרויקט)

1. בחרו מחלקה אחת מה-Class Diagram שלכם (למשל ישות כמו Order , או מחלקת קישור כמו OrderItem). הראו לי שהיא קיימת בפרויקט, ולא יזזו טבלה היא מתאימה.
2. בחרו use case אחד מהמפרט שלכם. הראו לי את המסך שמממש אותו, והריצו אותו מקצה לקצה: הזנת נתונים, ולידציה, וסיום הפעולה.
3. שתי ישויות קשורות זו לזו ב-Class Diagram. הראו לי איפה הקשר הזה מופיע בבסיס הנתונים (למשל ה-Foreign Key בין הטבלאות).

ב. מהקוד לתכנון (הכיוון הקשה יותר, לזהות את התכנון שלכם מתוך האפליקציה הרצה)

4. מסתכלים על המסך הזה באפליקציה הרצה. איזה use case זה? איזה user story הצדיק את בנייתו?
5. מסתכלים על המחלקה או הטבלה הזו בפרויקט. הצביעו עליה ב-Class Diagram וספרו לי מה היא מייצגת במערכת שלכם.

ג. פערים ושיקול דעת

6. הראו לי מקום אחד שבו התוצאה יצאה שונה מהתכנון שלכם, וספרו לי למה. (למשל שינוי שקלוד עשה, שדה שהוספתם בהמשך, תהליך שפישטתם, או מסך שיצא אחרת.)
7. האם היה משהו שחסר בתכנון המקורי שלכם וגיליתם אותו רק כשהפיתוח התחיל? למשל דרישה שפספסתם, ישות או שדה שלא חשבתם עליהם, או קשר בין נתונים שהתברר כנחוץ. מה זה היה, ואיך תיקנתם את התכנון?
8. אם הייתם צריכים להוסיף תכונה (feature) חדשה אחת למערכת מחר, מאיפה בתכנון הייתם מתחילים, ומה היה צריך להשתנות?

ד. הבנת ארכיטקטורה (ברמה הכללית, מושגים ולא קוד)

9. במילים פשוטות, מהם החלקים העיקריים של האפליקציה הזו ואיך הם מתחברים יחד? (מסכים, הנתונים שמוחזקים בזיכרון, בסיס הנתונים).

10. איפה הנתונים נמצאים, ובגדול מה צריך לקרות כדי שמשוהו שהמשתמש מקליד במסך יישמר בסוף בבסיס הנתונים? (בלי קוד, רק המסלול).

11. מהי מחלקה (class), ומה ההבדל בין מחלקה בדיאגרמה לבין טבלה בבסיס הנתונים?

בנוס: מעבר למה שהתבקש (רשות)

אם הקבוצה שלכם בנתה יותר ממה שהפרויקט ביקש, ספרו לנו. יש על כך ניקוד בונוס (עד 10%, ויותר על עבודה יוצאת דופן באמת, לפי שיקול דעת הבוחנים).

זה לא חלק חובה בהגנה. אין שאלות קבועות על כך, ואתם לא מפסידים דבר אם אין לכם את זה. אבל אם הרחקתם לכת, זו ההזדמנות שלכם להציג זאת. דוגמאות למה שנחשב:

- חיבור לשירותים חיצוניים: קריאה ל-API אמיתי, שליחת מיילים או התראות, שרת MCP וכדומה.
- GUI עשיר ומלוטש יותר, מעבר למסכים הבסיסיים שלימדנו.
- פונקציונליות נוספת: תכונות, דוחות או תהליכים מעבר להיקף הנדרש.
- כל דבר אחר שאתם גאים בו ומראה יוזמה אמיתית.

כדי לקבל את הניקוד, פשוט היו מוכנים להדגים שזה עובד ולהסביר בקצרה מה הוספתם ולמה. כמו בכל השאר, עליכם להיות מסוגלים להצביע על המקום שבו זה משתלב בתכנון שלכם.

איך נראית מוכנות

אתם מוכנים כשאתם יכולים לשבת עם שלושת החלונות פתוחים, ולכל use case או ישות שנצביע עליהם למצוא אותם בתכנון שלכם, למצוא אותם בפרויקט, ולהסביר איך השניים מתחברים, כולל היכן הם לא בדיוק תואמים ולמה. אינכם צריכים להסביר איך הקוד עובד.

זו כל ההגנה. בואו לדבר על מה שבניתם.